

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 Total Marks:50

Annexure A

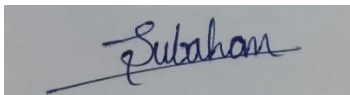
Scenario Based Assignment

Code of Conduct:

I T. Subahan (192511121) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'T. Subahan' written in a cursive style.

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty**
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic**
- iii. Analyze the impact of heuristic accuracy on both algorithms.**
- iv. Discuss how dynamic changes (blocked paths) affect search performance.**
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety**

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks:

- i. Explain how Hill Climbing would work in this scenario and identify its limitations.**
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation**

Local Maxima

- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations**
- iv. Recommend the best approach and justify your choice based on scalability and adaptability**

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search**
- ii. Challenges of Partial Observability and Dynamic Environments**
- iii. How the Rover Builds and Updates Its Internal Model**
- iv. Comparison of Online Search with Uninformed and Informed Search**
- v. Recommended: Online Search Agent**

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

- i. Clearly define variables, domains, and constraints with explanation.**
- ii. Explain how backtracking search works in this scenario**

Backtracking Search

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem**
- ii. Analyze the computational challenges of large game trees**

Computational Challenges of Large Game Trees

- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning**
- iv. Design an evaluation function and justify the factors considered**
- v. Recommend a strategy for balancing optimality and real-time performance**

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

Greedy Best-First Search (GBFS) Behavior

Greedy Best-First Search selects the next node based only on the heuristic value $h(n)$. It ignores the actual cost already traveled.

Behavior in this Scenario

- The drone always chooses the path that appears closest to the destination.
- It uses the straight-line distance heuristic to reach the affected area as quickly as possible.
- If a road becomes blocked, it changes direction and searches another path.

Strengths

- Very fast decision-making.
- Explores fewer nodes.
- Suitable when an immediate route is required.
- Low computational cost.

Weaknesses

- Does not guarantee the shortest or safest path.
- Ignores movement costs caused by weather or difficult terrain.
- May repeatedly encounter blocked paths.
- Can produce suboptimal routes.

ii. Explain how A* would handle the same situation, including how it balances cost and

heuristic

A* Search Behavior

A* Search evaluates each path using:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = Actual travel cost from the start.
- $h(n)$ = Estimated remaining cost (heuristic).

Behavior in this Scenario

- The drone considers both:
 - Actual travel cost (weather, terrain, fuel usage)
 - Estimated remaining distance.
- If heavy rain increases travel cost or a path becomes blocked, A* recalculates and searches for another lower-cost route.

Advantages

- Finds the optimal path if the heuristic is admissible.
- Avoids expensive or risky routes.
- Considers both distance and movement cost.
- Better suited for emergency delivery where safety matters.

iii. Analyze the impact of heuristic accuracy on both algorithms.

Heuristic Accuracy	Greedy Best-First Search	A* Search
Highly Accurate	Very fast and often finds a good path	Finds the optimal path efficiently
Moderately Accurate	May choose poor routes	Still usually finds the optimal path
Poor/Inaccurate	Performance decreases significantly; wrong decisions are common	Explores more nodes but still finds the optimal path if the heuristic does not overestimate

iv. Discuss how dynamic changes (blocked paths) affect search performance.

Effect of Dynamic Changes (Blocked Paths)

Greedy Best-First Search

- May repeatedly move toward blocked roads.
- Often needs to restart or backtrack.
- Route quality decreases in highly dynamic environments.

A* Search

- Updates path cost when obstacles appear.
- Recomputes the best available route.
- Produces safer and more reliable navigation.
- Although slower than GBFS, it adapts better to changing conditions.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Recommended: *A Search Algorithm**

Factor	Greedy Best-First Search	A* Search
Speed	Very High	High
Optimal Route	No	Yes
Considers Actual Cost	No	Yes
Handles Weather Cost	Poorly	Effectively
Handles Blocked Paths	Fair	Good
Safety	Moderate	High
Reliability	Moderate	High

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks:

i. Explain how Hill Climbing would work in this scenario and identify its limitations.

Hill Climbing in this Scenario

Hill Climbing is a local search algorithm that starts with an initial traffic signal configuration and continuously makes small changes that improve traffic flow.

Working

1. Start with current traffic signal timings.
2. Evaluate traffic performance.
3. Make a small adjustment (increase or decrease green-light duration).
4. If the new configuration reduces waiting time or congestion, accept it.
5. Repeat until no better neighboring solution exists.

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

Local Maxima

A solution that is better than neighboring solutions but is not the best overall.

Plateau

A region where many neighboring solutions have the same performance.

Ridge

A narrow path where improvement requires multiple coordinated changes.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Simulated Annealing

Simulated Annealing sometimes accepts worse solutions with a certain probability.

Working

- Begins with a high "temperature."
- Occasionally accepts poorer solutions to escape local maxima.
- Gradually reduces the temperature.
- Eventually settles on a near-optimal solution.

Genetic Algorithm (GA)

Genetic Algorithms imitate natural evolution.

Working

1. Generate many traffic signal timing solutions (population).
2. Evaluate each solution using a fitness function.
3. Select the best-performing solutions.

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Criteria	Hill Climbing	Simulated Annealing	Genetic Algorithm
Large Search Space	Poor	Good	Excellent
Escapes Local Maxima	No	Yes	Yes
Handles Dynamic Traffic	Limited	Good	Excellent
Scalability	Low	Medium	High
Global Optimization	No	Near-optimal	Near-optimal/Optimal
Adaptability	Low	Medium	High

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i.Explain why this problem requires an online search agent instead of offline search

Why This Problem Requires an Online Search Agent Instead of Offline Search

An online search agent makes decisions while interacting with the environment. The rover has no complete map of Mars and must discover the terrain as it moves.

Why Offline Search is Not Suitable

- Offline search requires complete knowledge of the environment before planning.
- The rover does not know the locations of rocks, craters, or obstacles in advance.

- New hazards are detected only during exploration.
- Precomputed paths may become unsafe or impossible.

Why Online Search is Suitable

- The rover senses its surroundings while moving.
- It continuously updates its knowledge of the environment.
- It chooses actions based on newly discovered information.
- It can safely react to unexpected obstacles.

ii. Challenges of Partial Observability and Dynamic Environments

1. Partial Observability

The rover cannot observe the entire Martian surface at once because its sensors have limited range.

Challenges

- Unknown obstacles may exist beyond sensor range.
- The rover has incomplete information.
- Decisions must be made with uncertainty.
- Risk of entering hazardous areas.

2. Dynamic Environment

Although Mars changes slowly, new conditions may arise during exploration.

Challenges

- Dust storms reduce visibility.
- Loose rocks may block the planned path.
- Sensor readings may change over time.
- Communication delays prevent immediate help from Earth.

iii. How the Rover Builds and Updates Its Internal Model

The rover maintains an internal map (world model) of its surroundings.

Working Process

1. Start with little or no map.
2. Move to a nearby location.
3. Use sensors (camera, LiDAR, radar, etc.) to observe the environment.
4. Detect hazards such as rocks, craters, or steep slopes.
5. Update the internal map with newly discovered information.

iv. Comparison of Online Search with Uninformed and Informed Search

Feature	Uninformed Search	Informed Search	Online Search
Environment Knowledge	None	Uses heuristic	Learns while exploring
Complete Map Required	Yes	Yes	No
Handles Unknown Terrain	No	Limited	Yes
Adapts to New Obstacles	No	Limited	Excellent
Real-Time Decision Making	Poor	Moderate	Excellent
Suitable for Mars Rover	No	Partially	Yes

v. Recommended: Online Search Agent

Criteria	Offline Search	Informed Search	Online Search
Unknown Environment	Poor	Moderate	Excellent
Partial Observability	Poor	Limited	Excellent
Dynamic Changes	Poor	Moderate	Excellent
Adaptability	Low	Medium	High
Safety	Moderate	High	Very High
Real-Time Decisions	No	Limited	Yes

Advantages of Online Search

- Operates without a complete map.
- Continuously learns from sensor data.
- Updates its internal model in real time.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

i. Clearly define variables, domains, and constraints with explanation.

Variables, Domains, and Constraints

1. Variables

Each course represents a variable.

Example:

- C1 = Data Structures
- C2 = Database Management System
- C3 = Artificial Intelligence
- C4 = Operating Systems
- C5 = Computer Networks

Domains

The domain is the set of possible values for each variable.

Example:

Time Slots

- T1 = Monday Morning
- T2 = Monday Afternoon
- T3 = Tuesday Morning

ii. Explain how backtracking search works in this scenario

Backtracking Search

Backtracking is a depth-first search technique that assigns values one variable at a time.

Working

1. Select the first course.
2. Assign an available time slot and hall.
3. Check whether all constraints are satisfied.
4. If valid, assign the next course.
5. If a conflict occurs, undo the previous assignment (backtrack).

Constraint Propagation (Forward Checking)

Constraint propagation reduces the search space by eliminating invalid choices early.

Forward Checking Process

After assigning a value to one course:

- Remove conflicting time slots from the domains of related courses.
- Detect conflicts before they occur.
- Stop searching if a variable has no remaining valid values.

Analyze real-world challenges and justify the best strategy for scalability

- **Real-World Challenges**
- Large number of students and courses.

- Limited exam halls.
- Faculty availability changes.
- Last-minute timetable modifications.
- Special accommodations for some students.

Conclusion:

The university exam scheduling problem is naturally modeled as a Constraint Satisfaction Problem (CSP), where variables represent courses, domains represent available time slots and halls, and constraints ensure there are no exam overlaps, hall conflicts, faculty scheduling issues, or accommodation problems. Backtracking search systematically.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

The Minimax algorithm is used in two-player competitive games where one player tries to maximize the score while the opponent tries to minimize it.

How It Works

- The AI is the MAX player, aiming to achieve the highest possible score.
- The human opponent is the MIN player, trying to reduce the AI's score.
- The game is represented as a game tree, where:
 - Each node represents a game state.
 - Each branch represents a possible move.

ii. Analyze the computational challenges of large game trees

Computational Challenges of Large Game Trees

In real-time strategy games:

- Every position has many possible moves (high branching factor).
- Each move creates additional possible future moves.
- The number of game states grows exponentially.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

Alpha-Beta Pruning is an optimization of the Minimax algorithm that avoids exploring branches that cannot affect the final decision.

Key Values

- Alpha (α): Best value found so far for the MAX player.
- Beta (β): Best value found so far for the MIN player.

Working

1. Evaluate nodes using Minimax.
2. Update α and β values.

iv. Design an evaluation function and justify the factors considered

Factors Considered

1. Resources

- More resources allow stronger future actions.

2. Army Strength

- Stronger armies increase the chance of victory.

3. Map Control

- Controlling key areas provides strategic advantages.

4. Building Health

- Protecting important structures is essential.

5. Enemy Strength

- Strong opponents reduce the evaluation score.

v. Recommend a strategy for balancing optimality and real-time performance

- ☐ Minimax for optimal decision-making.
- ☐ **Alpha-Beta Pruning** to eliminate unnecessary branches.
- ☐ A **well-designed evaluation function** to estimate non-terminal game states.